

Stopping LLM Agents from Damaging Production Systems

Executive briefing - 5 June 2026

This system prevents an LLM agent from taking an action that harms a production system. It sits between the model and the things it can touch - the shell, the network, and the kdb+ data store - and makes a deterministic allow / block / human-approval decision on every action, before the action runs. The model cannot talk its way past it.

1. The problem: an LLM with tools can break production

Give an LLM agent the ability to run commands and query data, and a single wrong tool call can:

- drop or delete a kdb+ table, or run an unbounded query that pulls an entire HDB into memory and stalls the tickerplant during market hours;
- exfiltrate client positions, P&L, or other classified data to an outside host;
- delete files, force-push, or touch a production system it was never meant to reach.

Prompt injection makes it worse: instructions hidden in data the agent reads can hijack the model into issuing exactly such a call. Telling the model 'do not do that' is advice it can ignore. The control must live outside the model and decide before the action executes.

2. How it stops them - two enforcement points the LLM cannot bypass

A. The tool-use gate (the enforcement engine)

Every time the LLM goes to use a tool - run a command, write a file, call an MCP tool, execute a query - a hook fires and the engine decides allow / block / require-human-approval before the action executes. The engine, not the model, decides whether the action happens.

- In Claude Code, it runs as a PreToolUse hook on every tool call.
- Via any model API (Anthropic, OpenAI, Bedrock), a broker intercepts each tool the model requests before the application executes it.
- In production, the engine runs as an out-of-process service the agent cannot tamper with; if it is unreachable, every action is denied (fail-closed).

The engine is default-deny: the agent can do only what the security team explicitly granted - specific tools, commands, writable paths, network destinations, and queryable tables. The policy is a signed file owned by the control function; because it is cryptographically signed and mounted read-only, the agent cannot edit or forge its own guardrails. Each MCP server is zero-privilege until a manifest explicitly grants it tools.

B. The kdb+ proxy (interface between the agent / MCP and the kdb+ system)

The agent never gets a direct handle to kdb+. Its queries pass through the query proxy, which sits between the MCP / agent layer and the kdb+ data store and enforces on the real query text:

- it parses the q / SQL;
- it injects a date-partition filter and a row cap, so a full-table scan cannot pull an entire HDB into memory and stall the tickerplant;
- it blocks mutations (delete / drop / update) - the interface is read-only by default;
- it rejects non-allowlisted tables and anything it cannot safely parse (fail-closed).

So the kdb+ production system only ever receives a bounded, read-only, allowlisted query - a runaway or destructive query physically cannot reach it. A matching egress proxy sits between the agent and the network and blocks exfiltration of classified data even to an otherwise-allowlisted host.

What actually gets stopped

- **BLOCKED** - delete from trade / drop table: mutation; the interface is read-only.

- **REWRITTEN** - unbounded select from quote: date filter + row cap injected before kdb+ runs it.
 - **BLOCKED** - upload positions data to an external host: network egress + classified data.
 - **BLOCKED** - query aimed at the production tickerplant: production target.
 - **BLOCKED** - agent edits its own policy or a protected file: signed, read-only guardrails.
-

Every decision is written to a tamper-proof audit log - hash-chained, mirrored off-host, and anchored so deletion or truncation is detectable. This is the regulator-facing record of what the agent tried and what was allowed.

3. Why it cannot be ignored

- The decision is outside the model's control loop - made by a separate process before the tool runs.
- Default-deny - the agent can only do what was explicitly granted.
- Fail-closed everywhere - missing or forged policy, unreachable engine, unknown tool, or any internal error results in BLOCK, never a silent allow.
- Signed policy - the agent cannot disable or rewrite its own guardrails.
- Proven, not asserted - 15 of 15 automated checks pass from one command; the core property is established by an exhaustive formal proof; demonstrated gating a real model call live.

4. How this compares to the industry

The field has independently converged on this approach - deterministic, out-of-process, default-deny enforcement that does not trust the model. Classifier-style content guardrails have been shown evadable up to 100% (arXiv:2504.11168); standards bodies (CoSAI / OASIS, OWASP) now state plainly: never rely on the LLM for security-critical validation.

Production controls split into three planes, and no single competitor spans all of them: content filtering (e.g. AWS Bedrock Guardrails), tool-call authorization (e.g. AWS Cedar / AgentCore), and the data / query plane (e.g. Satori, Immuta, which rewrite SQL to inject row filters). This system unifies the tool-use gate, the query plane, network egress, signed audit, and confinement into one default-deny pipeline - and does the query-plane interface for kdb+ / q, which the SQL-oriented data-governance proxies do not support.

5. Compliance posture (primary-source verified)

- EU AI Act (binding for high-risk systems) maps directly: Art. 12 event logging -> the tamper-proof audit; Art. 14 human oversight -> the require-approval path; Art. 15 robustness / cybersecurity -> the red-team-tested, signed, fail-closed gate.
- SR 26-2 (Apr 2026) supersedes SR 11-7 and places generative / agentic AI outside formal model-risk-management scope - so this is the operational governance control such tools still require, not MRM compliance.

6. Status and honest limitations

- Implemented and tested today: the tool-use gate, the kdb+ query proxy, the egress proxy, MCP manifests, signed policy, tamper-proof audit, confinement validation, formal proof, and the assurance suite - 15/15 core checks green.
- It governs agent ACTIONS. It does not make the model truthful and does not replace the bank's identity, data-loss-prevention, or change-control systems.
- Guarantee by surface: containerised operational agents get the strongest posture; developer laptops get strong defense-in-depth (a laptop cannot be fully sandboxed).
- Planned: an official end-to-end adversarial benchmark run, a machine-checked formal proof, and production audit-sink and human-approval back-ends.

Prepared from two adversarially-verified research passes plus primary-source verification. Not legal advice; validate regulatory mappings with your control function.